Functional Analysis of Darwin Sea Level Pressure (1951-2025)

Sanjib Samadder | Data Scientist

Contents

Loading Dataset and Initial Setup **2**

Task 1 :Implement Functional Smoothing Techniques. **2**

Setting up data structure & smoothing parameter 2

Setting up the plots and choosing best λ vlaue for propepr fit 2

Explanation of my chosen λ 4

Task 2: Functional Descriptive Statistics **4**

Task 2(i): Elementary pointwise mean and standard deviation of Darwin SLP 4

Justification of my Mean and Standard Deviation 5

Task 2(ii): Bivariate Covariance Function 5

Justification : Bivariate Covariance Function 6

Task 2(iii): Contour plot of the variance surface 6

Justification : Contour plot 7

Task 2(iv): Phase-Plane plots 7

Justification : Phase-Planeplot 8

Task 3: Functional Principal Component Analysis (FPCA) **8**

Task 3(i): Scree plot to determine number of principal components 8

Justification: Scree plot 9

Task 3(ii): Calculate eigenvalues and variance proportions 10

Justification : Eigenvalues and variance proportions 11

Task 3(iii): VARIMAX rotation technique with eigenvalues 11

Description : VARIMAX rotation 13

Loading Dataset and Initial Setup

```
library(fda)
data <- as.matrix(read.table("data/fda-SLP.dat"))
```

Task 1 :Implement Functional Smoothing Techniques.

Setting up data structure & smoothing parameter

```
darwin_slp_by_month = t(data) # Transpose to 12 months × 75 years

timeRng = c(1, 12)
time = seq(1, 12, 1)
nbasis = 12
Tempbasis = create.fourier.basis(timeRng, nbasis)
harmaccelLfd = vec2Lfd(c(0, (2*pi/diff(timeRng))^2, 0), timeRng)

# Determine smoothing parameter   by GCV
loglam = seq(-2, 5, 1)
nlam = length(loglam)
dfsave = rep(NA, nlam)
names(dfsave) = loglam
gcvsave = dfsave
invisible({
  for (ilam in 1:nlam) {
    lambda = 10^loglam[ilam]
    fdParobj = fdPar(Tempbasis, harmaccelLfd, lambda)
    smoothlist = smooth.basis(time, darwin_slp_by_month, fdParobj)
    dfsave[ilam] = smoothlist$df
    gcvsave[ilam] = sum(smoothlist$gcv)
  }
})
```

Setting up the plots and choosing best λ vlaue for propepr fit

```
par(mfrow = c(2, 2), mar = c(4, 4, 3, 2))

# GCV for smoothing parameter selection
plot(loglam, gcvsave, type='b', lwd=2,
     ylab=expression(GCV(lambda)),
     xlab=expression(log[10](lambda)),
     main="GCV for Smoothing Parameter Selection")
abline(v=0, col="red", lty=2)

# Smoothing with   = 1 (10^0) as GCV graph shows global minimum value at 0
lambda = 10^0
fdParobj = fdPar(Tempbasis, harmaccelLfd, lambda)
```

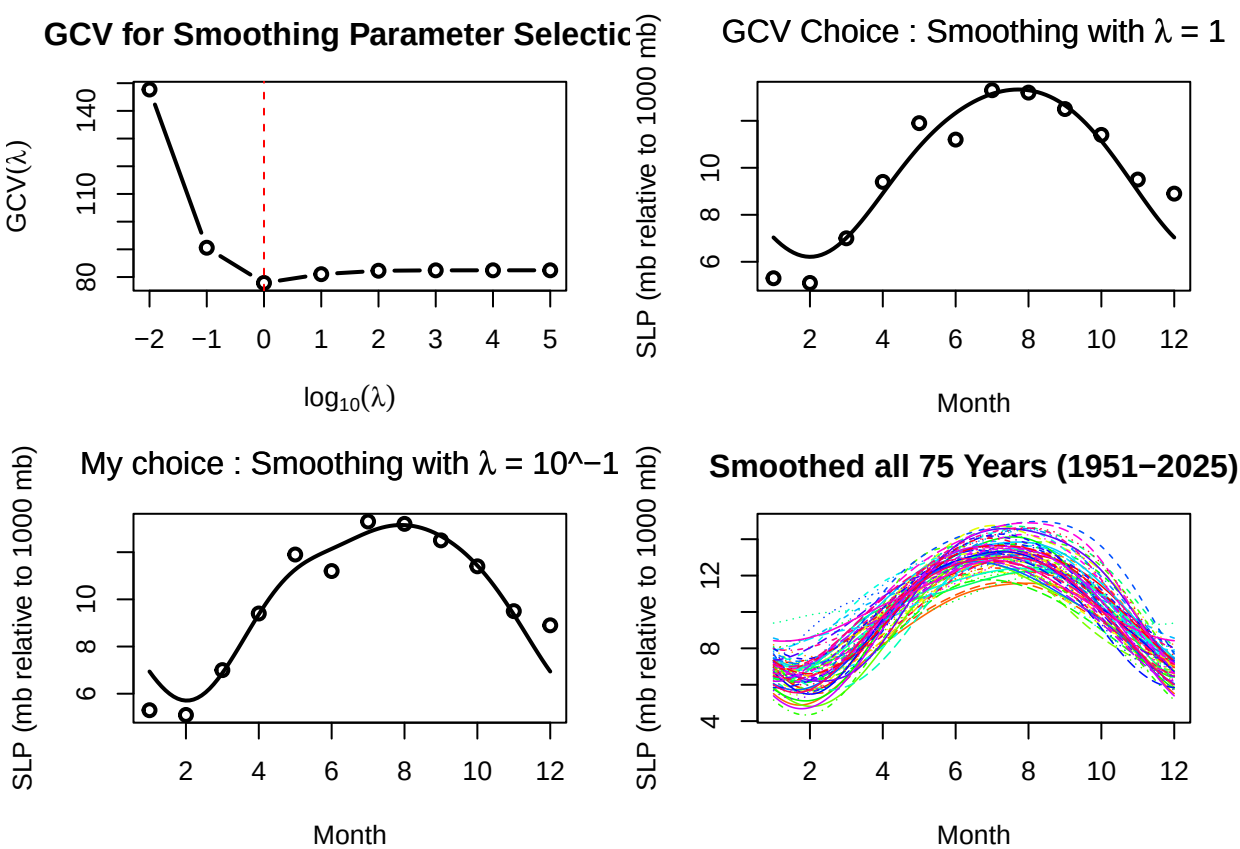
```r
darwin_slp_fit = smooth.basis(time, darwin_slp_by_month, fdParobj)
darwin_slp_fd = darwin_slp_fit$fd
fdnames = list("Month (1=January)", "Darwin SLP (mb relative to 1000 mb)", "Year")
darwin_slp_fd$fdnames = fdnames

plotfit.fd(darwin_slp_by_month[,1], time, darwin_slp_fd[1], lwd=2,
           main = expression(paste("GCV Choice : Smoothing with ", lambda, " = 1")),
           xlab = "Month", ylab = "SLP (mb relative to 1000 mb)")

# My choice : Smoothing with  = 0.1 (10^-1) as 10^0 was bit underfit
lambda = 10^-1
fdParobj = fdPar(Tempbasis, harmaccelLfd, lambda)
darwin_slp_fit = smooth.basis(time, darwin_slp_by_month, fdParobj)
darwin_slp_fd = darwin_slp_fit$fd
fdnames = list("Month (1=January)", "Darwin SLP (mb relative to 1000 mb)", "Year")
darwin_slp_fd$fdnames = fdnames

plotfit.fd(darwin_slp_by_month[,1], time, darwin_slp_fd[1], lwd=2,
           main = expression(paste("My choice : Smoothing with ", lambda, " = 10^-1")),
           xlab = "Month", ylab = "SLP (mb relative to 1000 mb)")

#All 75 smoothed curves
invisible(plot(darwin_slp_fd, lwd=0.8, col=rainbow(75),
     xlab="Month", ylab="SLP (mb relative to 1000 mb)",
     main="Smoothed all 75 Years (1951-2025)"))
```



```r
par(mfrow = c(1, 1))
```

Explanation of my chosen λ

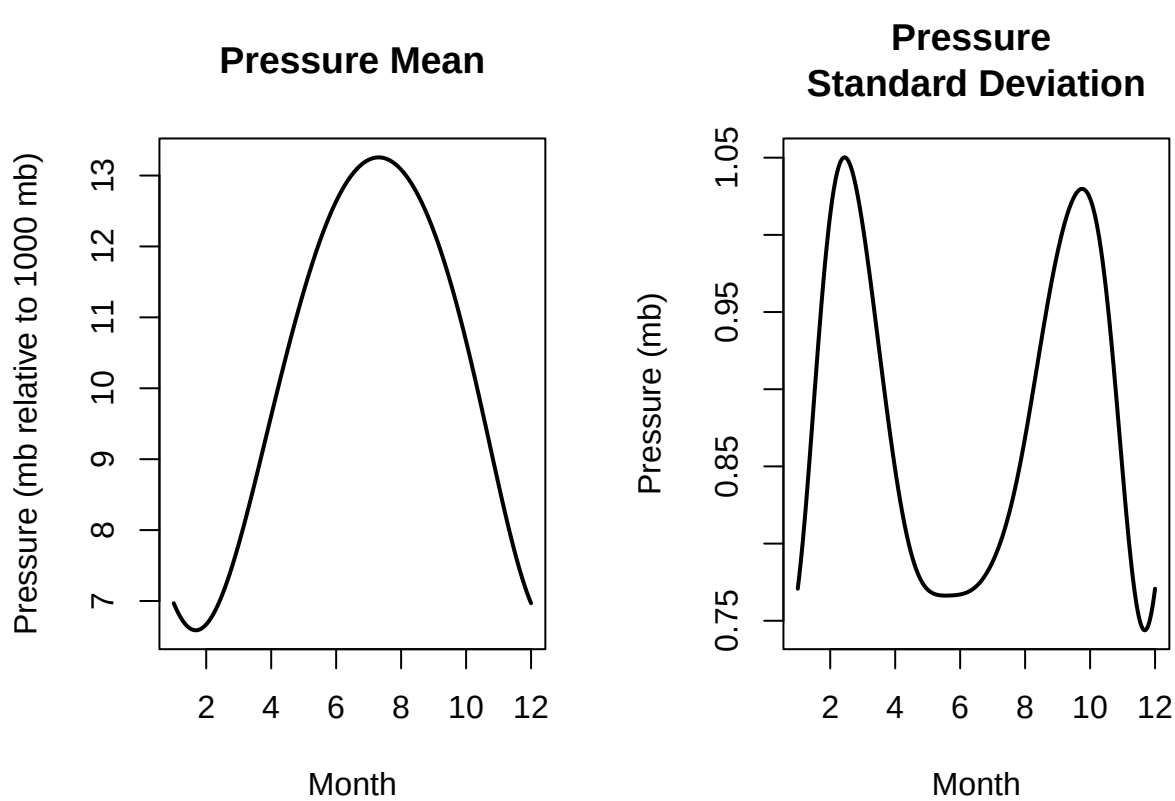
Initial GCV provides shows $\lambda = 10^0$ will be a good fit but after plotting the curve I see its bit under fit and missing some feature so I am taking $\lambda = 10^{-1}$ for better fit

Task 2: Functional Descriptive Statistics

Task 2(i): Elementary pointwise mean and standard deviation of Darwin SLP

```r
mean_slp = mean.fd(darwin_slp_fd) # calculating mean
stddev_slp = std.fd(darwin_slp_fd) # creating standard deviation function

# Creating plots
par(mfrow = c(1, 2))
invisible(plot(mean_slp, lwd=2, xlab="Month", ylab="Pressure (mb relative to 1000 mb)",
     main="Pressure Mean"))
invisible(plot(stddev_slp, lwd=2, xlab="Month", ylab="Pressure (mb)",
     main="Pressure \nStandard Deviation"))
```



```r
par(mfrow = c(1, 1))
```

Justification of my Mean and Standard Deviation

Mean

- From the mean curve, we can see in Jan to April pressure is low around 7/8 mb
- Highest pressure is in July to August around 12/13 mb
- This seasonal pattern reflects the influence of the Australian monsoon and the migration of the South Pacific High

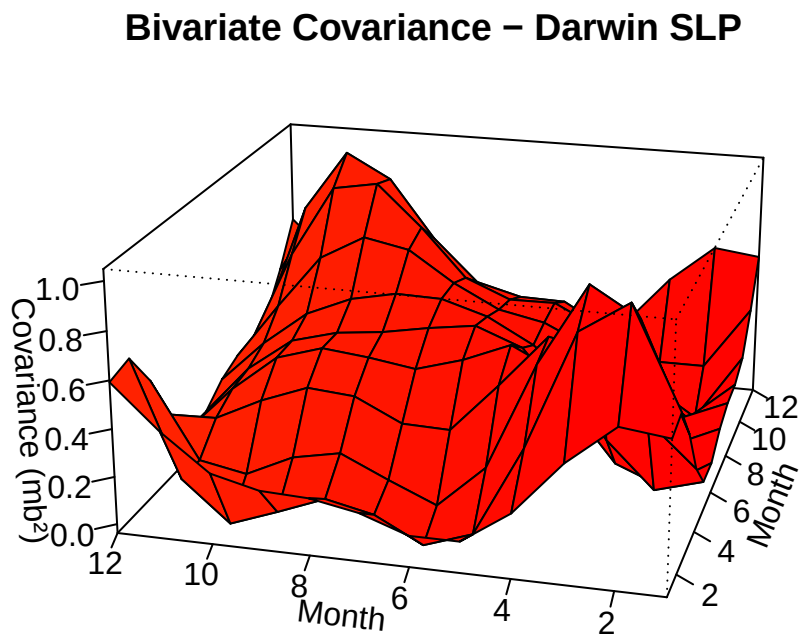
Standard Deviation

- The standard deviation function reveals interannual variability
- Higher variability in summer (Australia summer is Aug to Feb) - around 1 mb where in winter (Jun - Sep) low around 0.75
- This indicates that ENSO-related pressure anomalies are strongest during the Southern Hemisphere summer

Task 2(ii): Bivariate Covariance Function

```r
slp_covariance = var.fd(darwin_slp_fd) #bivariate covariance
slp_covariance_mat = eval.bifd(time, time, slp_covariance) #Evaluate covariance matrix at monthly point

# 3D perspective plot of covariance surface
persp(time, time, slp_covariance_mat,
      theta=-75, phi=20, r=7, expand=0.5,
      ticktype='detailed',
      xlab="Month", ylab="Month",
      zlab="Covariance (mb²)", col=rainbow(5000),
      main="Bivariate Covariance - Darwin SLP")
```

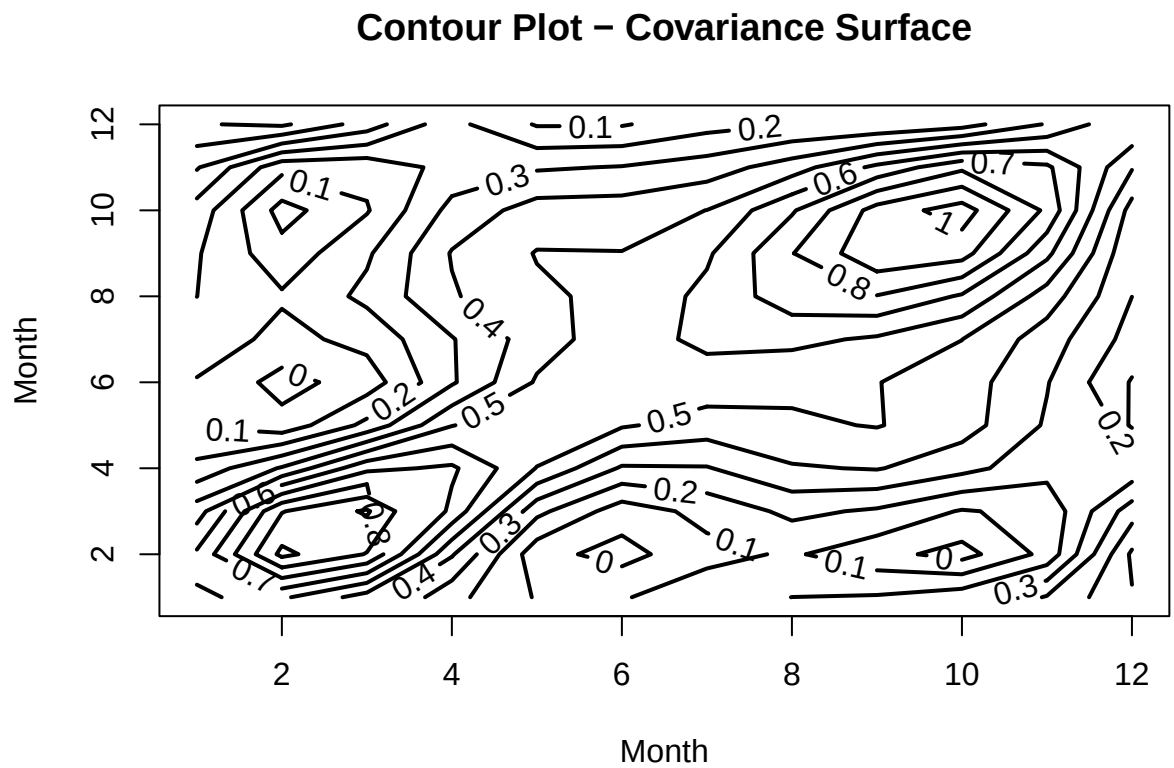


Justification : Bivariate Covariance Function

The 3D covariance surface displays the variance/covariance structure of Darwin SLP data with respect to months. Along the diagonal of the matrix, the covariance values represent the variances of the months, with higher variances in summer months (December and January) and lower variances in winter months (June and July). If the profile is cut vertically along the diagonal of the matrix, the profile is similar in shape to the standard deviation function.

Task 2(iii): Contour plot of the variance surface

```
# Contour plot of covariance matrix
contour(time, time, slp_covariance_mat,
        xlab="Month", ylab="Month", lwd=2, labcex=1,
        main="Contour Plot - Covariance Surface")
```

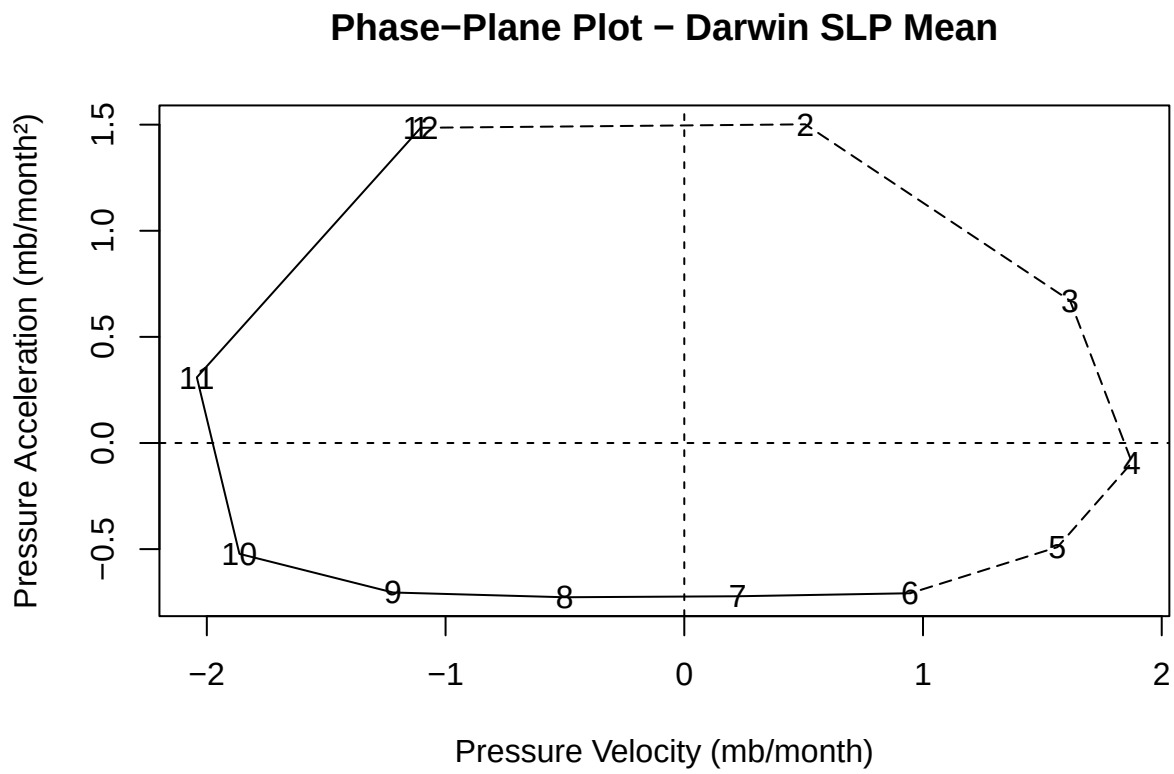


Justification : Contour plot

Contour lines locate features corresponding to the variance surface, running parallel to the diagonal and indicating approximate stationarity. The highest variance, shown by the most concentrated contours, occurs during summer months (December-January), matching the standard deviation function.

Task 2(iv): Phase-Plane plots

```
# Phase-plane plot for the mean function
phaseplanePlot(time, mean_slp,
               labels=list(evalarg=time, labels=1:12),
               xlab='Pressure Velocity (mb/month)',
               ylab='Pressure Acceleration (mb/month²)',
               main="Phase-Plane Plot - Darwin SLP Mean")
```



Justification : Phase-Planeplot

As we can see from our plot, start at month "1" (January) in the middle left and move clockwise through the 12 months, following the circular trajectory. The cycle sarts with moderate negative velocity, after that it goes to peak positive velocity around April-May, indicating rapid pressure increase during autumn. Maximum pressure occurs around July-August (months "7-8") where velocity approaches zero. The cycle then goes to negative velocity, with peak negative velocity around October-November (months "10-11"), representing rapid pressure decrease during spring. Finally, month "12" closely aligns with month "1", confirming the periodic nature of the annual SLP cycle. The circular trajectory indicates harmonic motion.

Task 3: Functional Principal Component Analysis (FPCA)

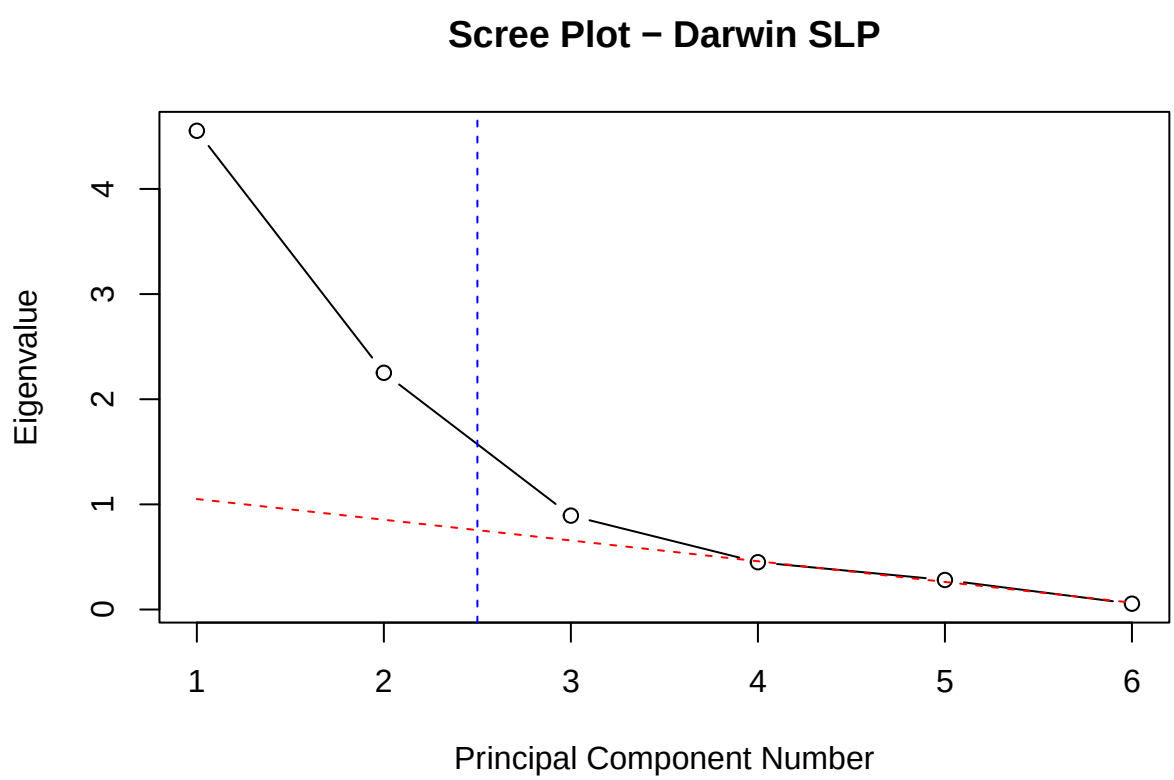
Task 3(i): Scree plot to determine number of principal components

```
# Preset number of PCs to evaluate
nharm = 3
darwin_pca_temp = pca.fd(darwin_slp_fd, nharm)

# Extract eigenvalues
fdaeig = darwin_pca_temp$values
neig = 6
```

```r
# Fit linear trend to later eigenvalues
x = matrix(1, neig - nharm, 2)
x[,2] = (nharm + 1):neig
y = as.matrix(fdaeig[(nharm + 1):neig])
c = lsfit(x, y, int = FALSE)$coef

# Scree plot
plot(1:neig, fdaeig[1:neig], "b",
     xlab = "Principal Component Number",
     ylab = "Eigenvalue",
     main = "Scree Plot - Darwin SLP")
lines(1:neig, c[1] + c[2] * (1:neig), lty = 2, col = "red")
abline(v = 2.5, col = "blue", lty = 2)
```



Justification: Scree plot

The first six eigenvalues obtained from the functional PCA on the Darwin SLP dataset are represented in the scree plot above. The dashed line represents the linear trend in the last four eigenvalues in the sequence, i.e., PC3 to PC6.From the scree plot above, we can see that the first two eigenvalues are significantly higher than the linear trend in the next four eigenvalues. This shows the importance of the first two principal components. We will select 2 principal components for our analysis.

Task 3(ii): Calculate eigenvalues and variance proportions

```r
# Select 2 principal components based on scree plot
nharm = 2
darwin_pca = pca.fd(darwin_slp_fd, nharm)

# Display eigenvalues
print(darwin_pca$values[1:2])
```

```
## [1] 4.553020 2.251256
```

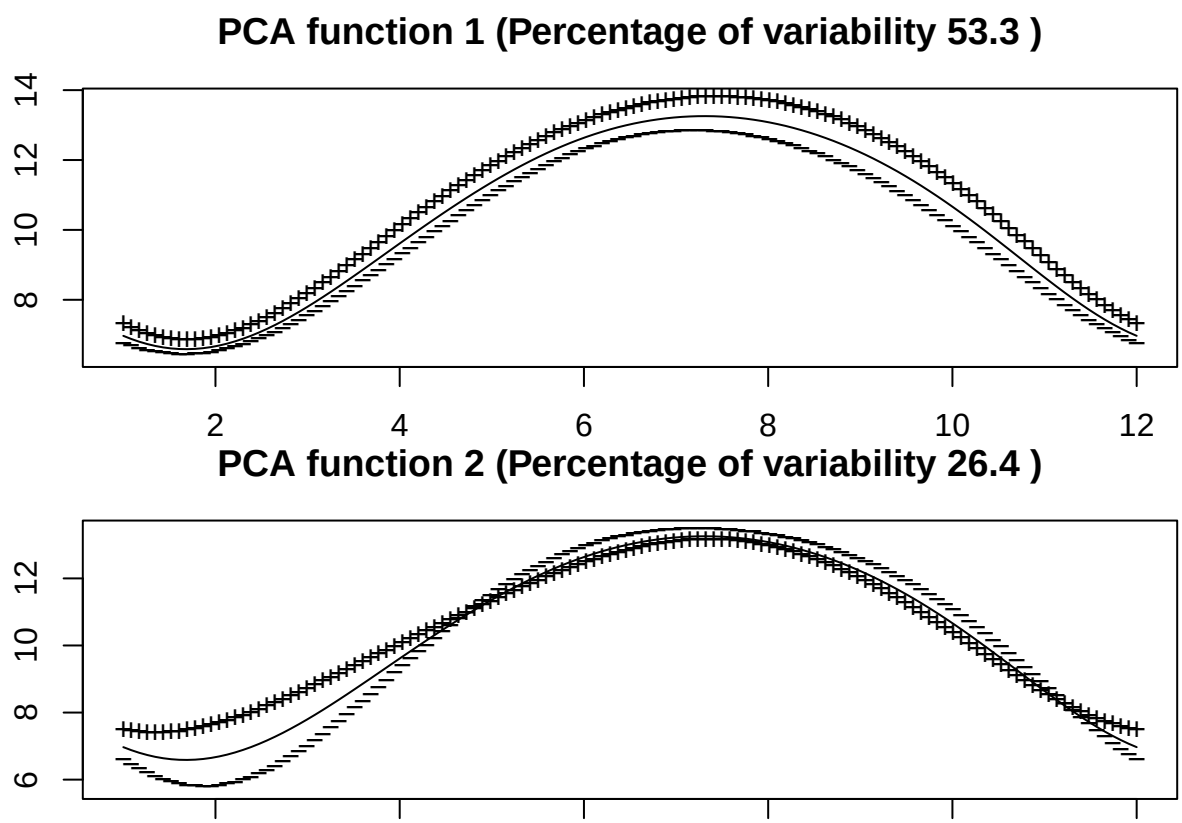
```r
cat(paste("PC1:", round(darwin_pca$varprop[1] * 100, 2), "%\n"))
```

```
## PC1: 53.34 %
```

```r
cat(paste("PC2:", round(darwin_pca$varprop[2] * 100, 2), "%\n"))
```

```
## PC2: 26.37 %
```

```r
# Plot both principal components in same horizontal line
par(mfrow = c(2, 1), mar = c(1, 2, 3, 2))
plot.pca.fd(darwin_pca, expand = 1.5)
```



```r
par(ask = FALSE)
par(mfrow = c(1, 1))
```

Justification : Eigenvalues and variance proportions

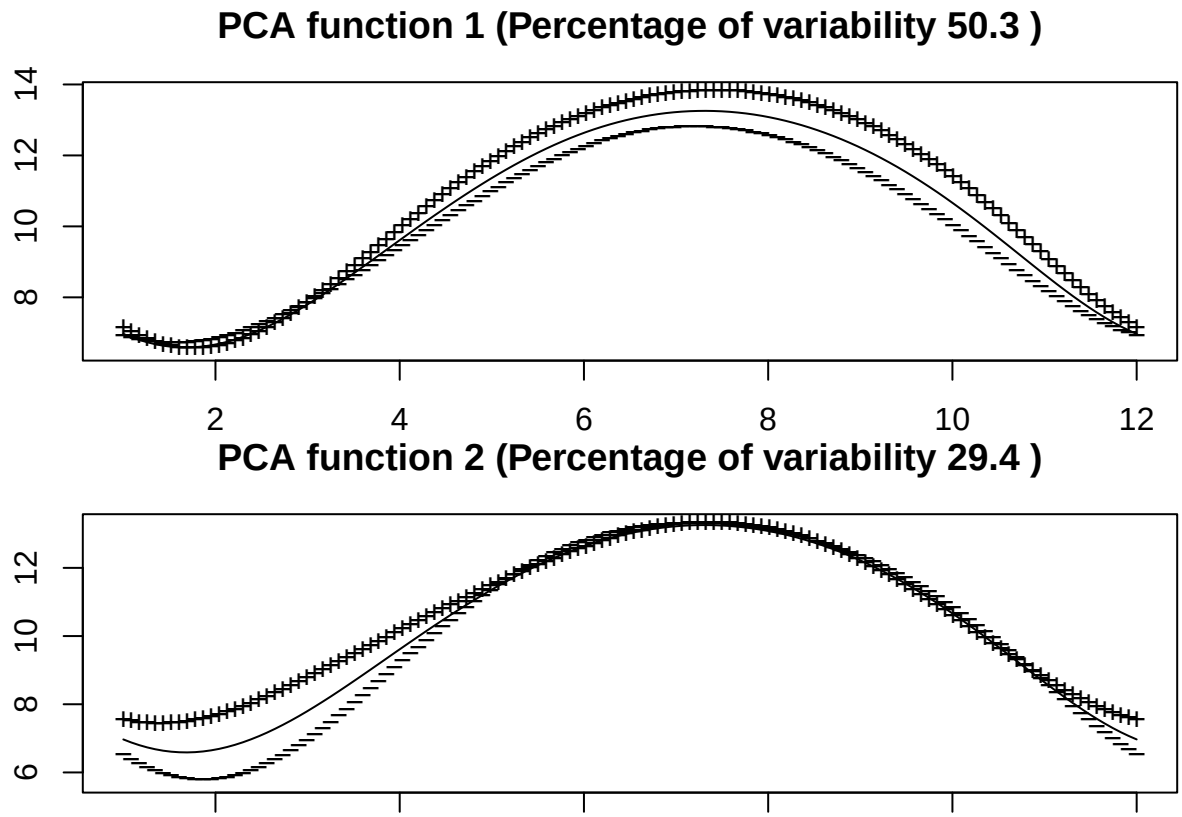
- The two main components are presented as a perturbation from the mean, where the mean is represented by the solid line.From the R code, we can see that the first two eigenvalues are 4.55 and 2.25, respectively.
- The first two components explain 53.34% and 26.37% of the total variation, respectively.Collectively, these first two components can explain up to 79.71% of the total variation.
- The figure above represents the unrotated functional principal components. The first functional principal component indicates a constant shift in the mean, which represents the overall pressure anomalies in all months. Negative values indicate low pressure, or El Niño conditions, whereas positive values indicate high pressure, or La Niña conditions.
- The second functional principal component indicates the contrast between summer and winter months. It represents the timing or strength of the annual cycle in the pressure anomalies.

Task 3(iii): VARIMAX rotation technique with eigenvalues

Plots are on the next page

```r
# Apply VARIMAX rotation
darwin_pca_rotated = varmx.pca.fd(darwin_pca)

# Plot rotated principal components
par(mfrow = c(2, 1), mar = c(1, 2, 3, 2))
plot.pca.fd(darwin_pca_rotated, expand = 1.5)
```




```r
par(ask = FALSE)
par(mfrow = c(1, 1))
# Display rotated eigenvalues
print(darwin_pca_rotated$values[1:2])
```

```
## [1] 4.291875 2.512954
```

```r
# Display rotated variance proportions
print(darwin_pca_rotated$varprop)
```

```
## [1] 0.5027230 0.2943516
```

Description : VARIMAX rotation

- The two rotated principal component functions are given below. These are plotted as a function of perturbation from the mean function, which is given by the solid line.
- From the results obtained using R, we find that the first two eigenvalues are 4.29 and 2.51, respectively. This is not surprising because these two principal components explain 79.71 percent of the variation. Then we have the plot of the rotated functional principal components. The first component explains 50.27 percent of the variation. This component is associated with the summer ENSO signal. Negative values indicate low pressure systems (El Niño-like), while positive values indicate high pressure systems (La Niña-like) occurring in the austral summer season. On the other hand, the second component explains 29.44 percent of the variation. This component is associated with the winter season. This is because this component is associated with the Australian monsoon and winter pressure systems.
- The VARIMAX rotation is used to simplify the principal component structure. This is because this rotation separates the summer ENSO signal and winter seasonality.